



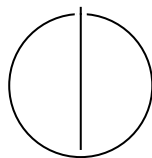
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY,
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Analysis and Optimization of LLM
Inference on Fast NVMe Storage**

Ersjan Keri





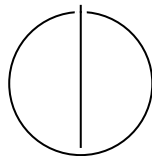
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY,
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Analysis and Optimization of LLM Inference on Fast NVMe Storage

Author:	Ersjan Keri
Examiner:	Prof. Dr. Viktor Leis
Supervisor:	Gabriel Haas
Submission Date:	10.05.2026



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 10.05.2026

Ersjan Keri

Acknowledgments

I am deeply grateful to my advisor, Gabriel Haas, whose guidance shaped this thesis from start to finish. He suggested the `io_uring` direction during an early discussion about the page-fault path, and that single pointer defined the entire optimization chapter: the application-managed `0_DIRECT` loader, the per-expert cache, the projection-group overlap, and the per-expert async dispatcher all descend from it. Beyond ideas, his careful reading and steady feedback across many iterations kept the work honest and the writing focused.

I thank my supervisor, Prof. Dr. Viktor Leis, for taking on the thesis.

I am grateful to the chair of Prof. Leis for providing the `cli-hiwi-02` server, arranged through Gabriel Haas, which hosted every cgroup sweep and every overnight run reported here; the AMD Ryzen 7 7700X and its two NVMe SSDs are the silent co-authors of every wall-clock number in the tables.

Finally, I thank my family for their patience through a project whose interesting moments were mostly invisible from the outside.

Abstract

A language model's weights are large arrays of numbers, called tensors, that the model multiplies on every generated token. The full set of tensors lives on disk in a single file, and the model runs fastest when the whole file is loaded into RAM. RAM has become expensive. DDR5 contract prices have roughly doubled since early 2025, and AI accelerator demand is forecast to consume about 20% of global DRAM supply in 2026. NVMe SSDs are cheap and fast. Our goal is to extend llama.cpp so that a large model can run on a machine that does not have enough RAM to hold the whole model.

We started our analysis by building a tensor-level tracker that records, for every operation the model performs, which tensor it reads and which region of memory the data comes from. The trace shows that only 26.91% of GPT-OSS-20B's weights and 7.69% of GPT-OSS-120B's weights are read per generated token, because each Mixture-of-Experts layer activates only 4 of its 32 experts on 20B, and 4 of 128 on 120B. The inactive experts, which make up most of the model file, can stay on the SSD. Our backend pins the always-needed weights (the attention, output, and norm tensors) in RAM so the operating system cannot evict them, and prefetches the expert weights from the SSD asynchronously, overlapping the disk reads with the matrix multiplications that consume earlier reads. GPT-OSS-20B (12.85 GiB on disk) runs at 14.32 tokens per second on our test machine when the whole model fits in RAM. With limited RAM, where the model no longer fits, the default llama.cpp loader drops to 1.44 tokens per second. Our backend, on the same setting, runs the same model at 5.36 tokens per second. GPT-OSS-120B (60.88 GiB on disk) does not fit in the 30 GiB of RAM on our test machine at all. With limited RAM the default loader runs it at 4.96 tokens per second and our backend at 8.06 tokens per second.

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
2. Background	3
2.1. Transformer Language Models	3
2.2. Mixture-of-Experts	3
2.3. How llama.cpp Loads Weights	6
2.4. Asynchronous Disk I/O on Linux	6
3. Related Work	8
3.1. Memory-Constrained Single-Host LLM Inference	8
3.2. Mixture-of-Experts Inference Systems	9
3.3. Asynchronous Storage I/O	10
3.4. Positioning	10
4. Tensor Tracing	12
4.1. The Tracer	12
4.2. What the Trace Reveals	12
4.3. Visualization	13
4.4. Simulating Replacement	14
5. Implementation	17
5.1. Pinning the dense weights	17
5.2. The expert buffer	17
5.3. Connecting the buffer to compute	18
5.4. Pipelining I/O with compute	18
5.4.1. Projection-group overlap	18
5.4.2. The fused MoE op	19
5.4.3. Async-projection-overlap	19
5.4.4. Async-experts	19

6. Evaluation	22
6.1. Setup	22
6.2. GPT-OSS-20B	23
6.3. GPT-OSS-120B	27
6.4. Cache eviction policy at undersized cache	27
6.5. Comparing the overlap modes	28
7. Discussion and Conclusion	29
7.1. Limitations	29
7.2. Future work	29
7.3. Conclusion	30
A. Software Artifacts	32
A.1. Forked llama.cpp	32
A.2. Tensor-Tracing Toolchain	33
A.3. Experiment Harness	35
A.4. Datasets and Result Archives	36
A.5. Reproducing the Headline Numbers	36
Abbreviations	38
List of Figures	39
List of Tables	40
Bibliography	41

1. Introduction

A modern language model’s weights are tensors, large arrays of numbers that the model multiplies on every generated token. The two models studied in this thesis, GPT-OSS-20B and GPT-OSS-120B [Ope25], hold 12.85 GiB and 60.88 GiB of tensors on disk against the 30 GiB of RAM on our test machine. The standard approach to inference, taken by llama.cpp [Gl23] among others, keeps the full set of model tensors resident in RAM throughout generation. RAM has become an expensive resource: DDR5 contract prices have roughly doubled since early 2025 [Int25; Swa25], and AI accelerator demand is forecast to consume about 20% of global DRAM supply in 2026 [Tre25]. This thesis investigates inference under limited RAM, with the bulk of the model file left on the SSD.

We built a tensor-level tracer that records which tensors each operation accesses over the course of generation. The trace shows that tensor access is not uniform. The dense per-layer tensors (attention, output, and the norms) are read on every generated token. The experts of each Mixture-of-Experts layer [Sha+17], which form the bulk of the model file, are not: a small router selects only a few of them per layer per token and the rest are skipped. The dense tensors and the experts needed for one generated token together account for 26.91% of GPT-OSS-20B and 7.69% of GPT-OSS-120B. Most of the model is therefore not needed at any one moment.

We pin the dense per-layer tensors in RAM so the kernel cannot evict them. We manage the experts ourselves in a dedicated buffer that reads them from the SSD asynchronously.

We additionally built a small simulator that replays the trace under simple eviction strategies (LRU, LFU, and a frequency-with-decay variant) and reports the resulting hit rate of the buffer for a given number of expert slots. The aim is to lay a foundation, not to design the best possible policy. Every measurement runs inside a kernel-enforced memory limit set with Linux cgroups [Heo15], so the available RAM is a hard, reproducible number rather than a condition that drifts between sessions.

With 7 GiB of RAM available, the default llama.cpp loader runs GPT-OSS-20B at 1.44 tokens per second. Our backend runs it at 5.36 tokens per second, almost four times faster (Figure 1.1). GPT-OSS-120B does not fit in 30 GiB of RAM at all. The default loader runs it at 4.96 tokens per second. Our backend runs it at 8.06 tokens per second.

1. Introduction

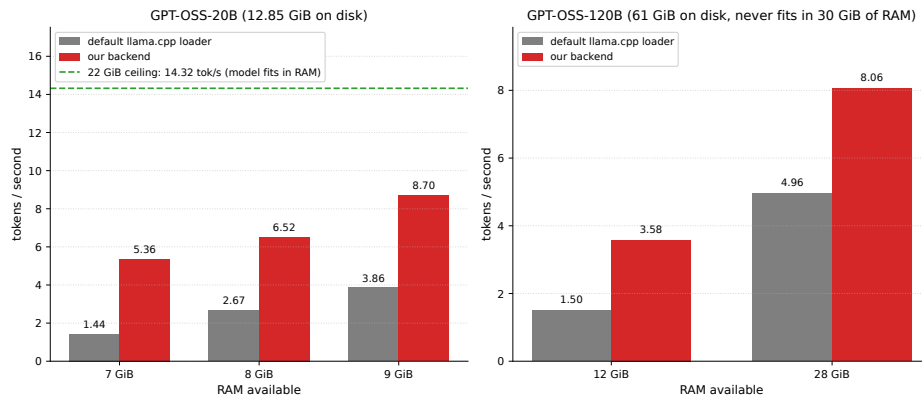


Figure 1.1.: Decode throughput of the default llama.cpp loader (gray) against our backend (red), at three RAM caps for GPT-OSS-20B and two for GPT-OSS-120B. The dashed line on the left panel marks the 14.32 tok/s ceiling reached when GPT-OSS-20B fits entirely in RAM at the 22 GiB cap. GPT-OSS-120B (60.88 GiB on disk) never fits in the 30 GiB of RAM on the test machine, so there is no in-RAM ceiling on the right panel.

2. Background

2.1. Transformer Language Models

A transformer language model is a stack of layers [Vas+17]. Each layer applies two sub-blocks in sequence: a *self-attention* block, in which each position attends to itself and earlier positions in the sequence, and a *feed-forward network* (FFN), which transforms each token’s representation independently. GPT-OSS-20B has 24 such layers; GPT-OSS-120B has 36 [Ope25].

Generating text happens in two phases. The first, called *prefill*, processes the entire prompt. The second, called *decode*, produces one new token at a time, each appended to the input for the next step. Our throughput metric counts tokens generated per second during decode.

To avoid recomputing attention over all prior tokens at every decode step, each layer caches its key and value tensors. We call this the *KV cache*. GPT-OSS interleaves layers using a 128-token sliding window with layers using full attention [Ope25], but the KV cache is allocated up front for the full configured 131072-token context. It therefore reserves 3.02 GiB of RAM on GPT-OSS-20B and 4.53 GiB on GPT-OSS-120B regardless of how many tokens are actually generated.

2.2. Mixture-of-Experts

In a Mixture-of-Experts layer, the FFN is replaced by many small subnetworks, called *experts* [Sha+17]. For each generated token, a small *router* selects only a few of them, and the rest are not read. GPT-OSS-20B has 32 experts per layer and selects 4; GPT-OSS-120B has 128 experts per layer and also selects 4 [Ope25].

Each expert is three weight matrices, called the *up*, *gate*, and *down* projections. We call each of these matrices a *slice*. A slice is 4.20 MiB on disk on both GPT-OSS models (4,406,400 bytes). One MoE layer therefore reads $4 \times 3 = 12$ slices per generated token. Across the 24 layers of GPT-OSS-20B that is 288 slices per token, 1.18 GiB of weight data. On GPT-OSS-120B it is $4 \times 3 \times 36 = 432$ slices, 1.77 GiB.

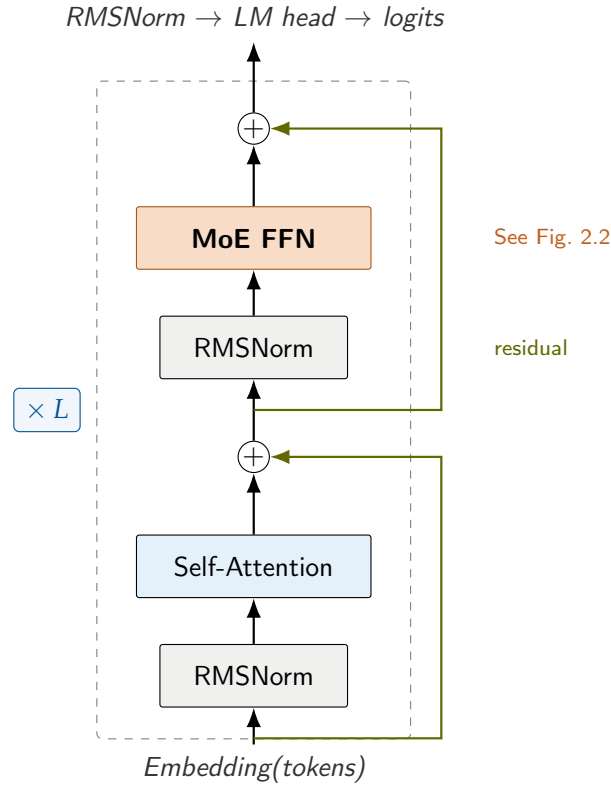


Figure 2.1.: One transformer layer. Self-attention and the feed-forward network are applied in sequence with residual connections; the model is a stack of L such layers ($L = 24$ for GPT-OSS-20B, $L = 36$ for GPT-OSS-120B) followed by a final norm and the output projection.

2. Background

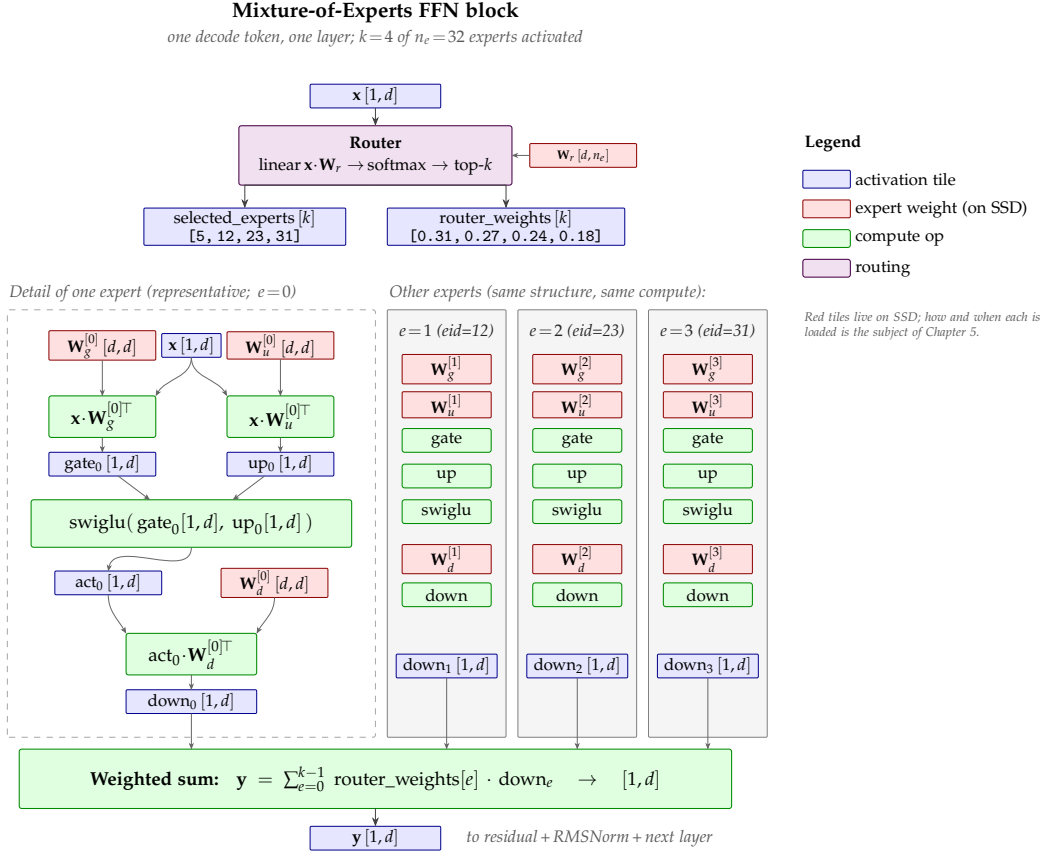


Figure 2.2.: One MoE layer applied to one generated token. The router selects 4 of the 32 experts (highlighted in red); the remaining 28 are not read. Each selected expert applies its three weight matrices (up, gate, down), and the four expert outputs are combined into the layer output.

2.3. How llama.cpp Loads Weights

llama.cpp [Gl23] stores model files in GGUF, a single-file binary format whose header lists each tensor’s name, shape, type, and offset, followed by the tensor data [Gg23].

The default loader maps the file into the process’s virtual address space with `mmap` [KL25b], so tensor bytes are read by dereferencing pointers. A first access to a not-yet-resident page triggers a *page fault*: the kernel suspends the thread, reads the page (and a small readahead window around it, on the order of tens of KiB) from disk into the *page cache*, maps it at the faulting address, and resumes the thread. Subsequent accesses to the same page return without entering the kernel. The page cache lives in kernel-managed memory, and the kernel chooses which pages to evict when other allocations need the space.

2.4. Asynchronous Disk I/O on Linux

`io_uring` is a Linux kernel interface for asynchronous I/O [Axb19]. The application and the kernel share submission and completion queues in memory: a *submission queue* (SQ), into which the application writes read requests, and a *completion queue* (CQ), into which the kernel writes results. A single `io_uring_enter` system call notifies the kernel that new submission entries are pending. The kernel reads them in place from shared memory and dispatches the I/O without blocking the calling thread. Each completion entry carries a user-supplied tag that identifies the request it satisfies, so the application can match completions to requests and drain the queue at its own pace.

`O_DIRECT` is an `open(2)` flag that tells the kernel to bypass the page cache for that file [KL25c]. On a read, the kernel programs the device to DMA the data directly into the application-supplied buffer with no copy through the page cache. The cost is a strict alignment contract: file offset, read length, and destination buffer address must all be multiples of the device’s logical block size, which is 512 bytes on the NVMe used here [NVM25]. The two features are orthogonal: `io_uring` can serve cached or direct I/O, and `O_DIRECT` can be used with synchronous `read(2)` as well.

Combining the two moves both the buffer and the eviction decision into the application. The submission and completion queues are `mmap’d` into the application’s address space at `io_uring` setup time, so the application writes submissions and reads completions without entering the kernel except for the one `io_uring_enter` call per batch. The data buffer is allocated by the application and is the DMA target (Figure 2.3). With no page cache between the device and the buffer, there is no kernel-side cache to evict from; if the application keeps a fixed-size pool of buffer slots, eviction is its responsibility (Figure 2.4).

2. Background

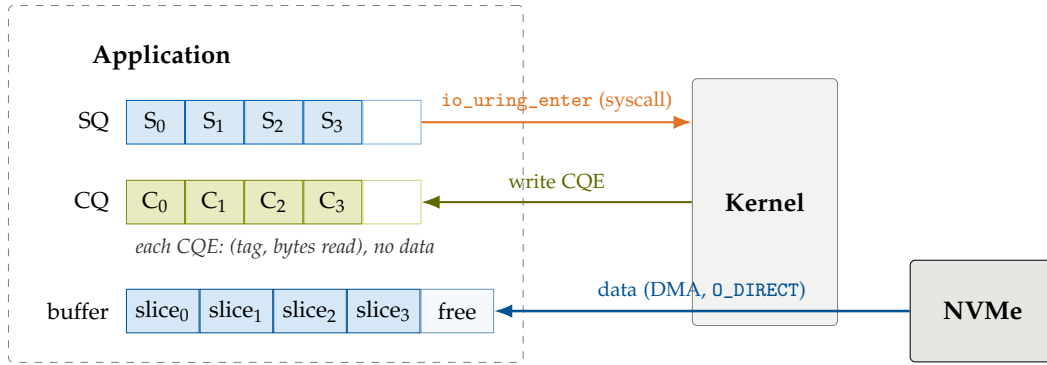


Figure 2.3.: Read flow under `io_uring` combined with `O_DIRECT`. The submission queue, completion queue, and the data buffer all live in the application's address space. Each SQE carries the destination buffer pointer for its read. One `io_uring_enter` system call (orange) notifies the kernel; the kernel reads the SQEs from shared memory, pins the destination pages, and programs the device. The device then DMA's the file bytes into the destination, overwriting whatever was there (blue): this is the only path on the figure that carries data. The kernel writes one CQE per read into the CQ ring (green); a CQE is a (tag, bytes-read) record only, no data. The application reads the CQ in place without another syscall.

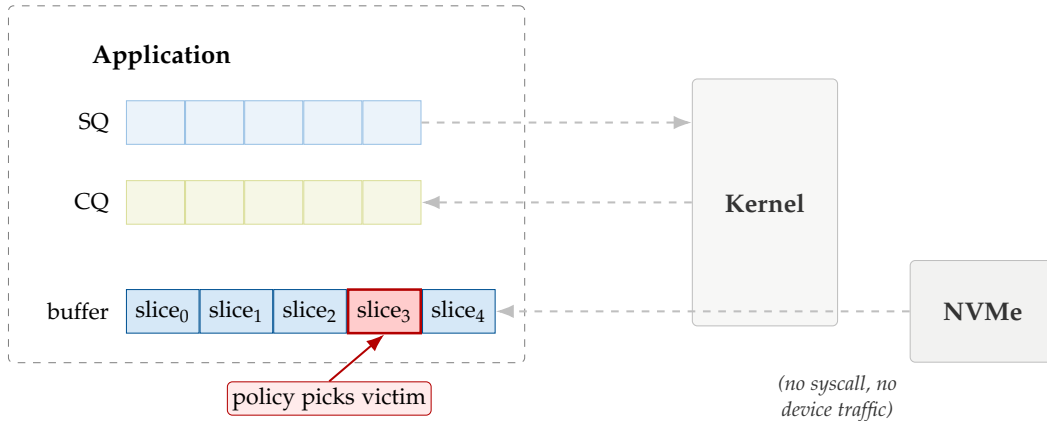


Figure 2.4.: Eviction step on a cache miss when the buffer is already full, drawn with the same building blocks as Figure 2.3. The application's replacement policy picks a victim slot (here `slice3`, in red); no system call and no device traffic occur. The freed slot then serves as the DMA target for the read flow of Figure 2.3.

3. Related Work

This work sits at the intersection of three lines of research: memory-constrained inference for dense language models, MoE-specific serving systems, and asynchronous storage I/O. The systems closest in deployment shape (a single host with the slow tier behind the CPU) target dense models with activation-level sparsity that GPT-OSS does not exhibit. The MoE-specific systems all assume either a multi-GPU cluster, a CPU-GPU host, or a GPU host with DRAM as the slow tier; none target single-host CPU-only with NVMe as the primary weight store.

3.1. Memory-Constrained Single-Host LLM Inference

LLM in a Flash [Ali+24] keeps a dense LLM on NAND flash and pages neuron weights into DRAM on demand. Two techniques carry the design. *Windowing* reuses neurons that fired in recent decode steps. *Row-column bundling* packs the weights of co-active neurons into contiguous chunks so each flash transfer is sequential rather than scatter-gather. The reported $4\text{--}5\times$ CPU and $20\text{--}25\times$ GPU speedups are against a naive on-demand baseline. The mechanism only works because the target models use ReLU FFNs with around 90% post-activation sparsity [Mir+24; Liu+23] and a small predictor identifies which neurons will fire next. GPT-OSS uses SwiGLU [Sha20; Ope25], which is gated rather than thresholded, and the sparsity LLM in a Flash exploits does not exist here.

ActiveFlow [Jia+25] addresses the gap LLM in a Flash leaves: dense LLMs with non-ReLU activations. Cross-layer active-weight preloading uses the current layer’s activations to predict next-layer weight needs. Sparsity-aware self-distillation fine-tunes the model so its post-activation pattern is more predictable. A dynamic DRAM-flash swap pipeline orchestrates the transfers at runtime. The submit-while-compute idea is the same shape as projection-group overlap (Subsection 5.4.1), but ActiveFlow’s predictability gain comes from retraining the model. For an off-the-shelf MoE such as GPT-OSS, retraining is out of scope, and per-layer routing is independent across layers (Section 2.2), so a learned cross-layer predictor would have to predict the router itself rather than read an activation pattern.

PowerInfer [Son+24] profiles which neurons are persistently activated across many tokens (“hot”) versus rarely activated (“cold”). The hot set is pinned on the GPU; the

cold neurons are computed on the CPU rather than transferred to the GPU. PowerInfer-2 [Xue+24b] extends the neuron-cluster decomposition to a smartphone NPU plus CPU. The hot/cold distinction is a function of activation-level sparsity in dense ReLU networks. In an MoE the equivalent question is which experts are frequently routed-to, but that is a workload-distribution property rather than an architectural one and shifts with prompt domain. The tier topology also differs from ours: we run on a CPU-only host with NVMe as the slow tier and no GPU on which to pin a hot set.

FlexGen [She+23] addresses high-throughput batched offline inference, where weights, KV cache, and activations are spread across GPU memory, CPU memory, and disk. An offline linear-program search picks a placement policy per tensor, and large batches amortize the I/O across many concurrent tokens. Our setting is the opposite end of the same spectrum: one in-flight request, decode latency as the metric of interest, no other concurrent tokens to amortize a single request’s expert reads against. FlexGen is the reference point for “offload everything everywhere” in the literature, but its scheduling techniques do not transfer to our regime.

3.2. Mixture-of-Experts Inference Systems

DeepSpeed-MoE [Raj+22] is the reference system for large-scale MoE inference on GPU clusters. The inference path supports expert parallelism (experts of each layer sharded across GPUs, with routed activations carried over the cluster network), tensor parallelism for the dense layers, and a pyramid-residual MoE compression scheme (PR-MoE) that shrinks the served model. Its deployment target is many GPUs with the full set of weights resident across aggregate device memory, network-bound rather than storage-bound. The expert-parallelism techniques do not apply when there is one host and one storage tier, but DeepSpeed-MoE establishes the baseline for how MoE is served at cluster scale.

Fiddler [Kam+25] is the closest MoE-specific design in spirit: single-host CPU–GPU MoE inference. A hot subset of experts stays resident on the GPU and the rest live in CPU RAM. For each routed expert, Fiddler dynamically picks between two execution paths: ship the weights from CPU RAM to the GPU and run the expert there, or ship the activations to the CPU and run the expert on the CPU. At small batch sizes the second path wins, because CPU compute is faster than the PCIe weight transfer plus GPU compute it would replace. The setting differs from ours in two ways that matter: Fiddler assumes a GPU is present, and the slow tier is CPU RAM rather than NVMe through the kernel block layer. The bandwidths and latency-hiding mechanics are different by an order of magnitude.

MoE-Infinity [Xue+24a] keeps experts in host DRAM (not on storage) and prefetches

them into GPU memory based on a request-similarity match. Each completed request leaves an Expert Activation Matrix (EAM) recording how often each expert fired at each layer. The in-flight request’s partial EAM, built up during the first few decode steps, is matched against historical EAMs in an EAM Collection by cosine similarity, and the closest match’s expert pattern drives the prefetch. GPU-side cache replacement is a dynamic priority scheme keyed on the predicted EAM, with protection of in-flight prefetches. Two differences position our work: MoE-Infinity’s slow tier is host DRAM at tens of GiB/s rather than NVMe at single-digit GiB/s, and the per-layer compute window on a GPU is short enough that cross-layer prediction is needed to win. Our per-layer compute window on a CPU is long enough that within-layer overlap suffices, and we prefetch nothing across layers or requests. The EAM construction itself is an interesting direction to extend our system in and appears as future work in Section 7.2.

3.3. Asynchronous Storage I/O

The two mature options for high-throughput storage I/O on Linux are `io_uring` with `O_DIRECT` [Axb19; KL25c] (used here, mechanics in Section 2.4) and SPDK [SPD26], a userspace NVMe driver that bypasses the kernel block layer entirely and polls the device from a dedicated thread. SPDK is the reference design for million-IOPS workloads where syscall overhead dominates. Our IOPS regime is bounded by per-layer 12-read batches at decode rates of a few tokens per second; total submissions per second sit in the low thousands, which the kernel block layer handles without measurable overhead. The deployment cost of SPDK (dedicated cores, hugepages, exclusive device access) would not be repaid by a workload that already saturates the NVMe at moderate queue depth.

3.4. Positioning

This thesis exploits a sparsity source that none of the prior systems above target directly: the architectural top- k routing of Mixture-of-Experts models [Sha+17; FZS22]. Routing is deterministic given the input, and per-layer routing is independent across layers, so no learned predictor and no model modification are required. The price for that generality is that cross-layer prefetching, the central trick in ActiveFlow and MoE-Infinity, is fundamentally harder for us, since predicting layer $N+1$ ’s routing means predicting the router itself rather than reading an activation pattern. We flag it as future work in Chapter 7. We are not aware of prior work that combines (a) MoE-architectural sparsity, (b) commodity NVMe as the primary weight store on a CPU-only host, (c)

3. *Related Work*

bit-exact application-managed I/O with workload-tuned cache policies, and (d) a kernel-enforced cgroup v2 benchmarking methodology.

4. Tensor Tracing

4.1. The Tracer

The tracer hooks into llama.cpp’s compute path at a single point that every operation crosses, so we capture every operation regardless of operator type. Only the first worker thread emits a record per operation. The other threads skip the trace path.

Each record is a fixed-size 1024-byte binary entry carrying the timestamp, the layer and token IDs, the destination tensor name, up to four source tensor descriptors, and the expert IDs for MoE-routed operations. Each source descriptor labels the source with a memory class: *disk* for the memory-mapped GGUF file (the model weights), or *buffer* for the runtime’s anonymous memory (KV cache and scratch). For disk-backed sources the descriptor also records the GGUF byte offset, so the access can be tied back to the file layout.

Records are written through a thread-local buffer that flushes into a memory-mapped log file, so writes never block compute. A small Python toolchain converts the binary log into per-token JSON, which feeds the visualizers.

A second, narrower instrumentation stream lives inside our expert buffer manager. It records the sequence of (layer, projection, expert) cache lookups together with the hit-or-miss outcome of each. This dump is the input to the offline replacement simulator, which replays it under alternative policies.

4.2. What the Trace Reveals

Each expert weight matrix on GPT-OSS-20B is 4,406,400 bytes on disk, or 4.20 MiB. Of the 12.85 GiB of model weights, 9.48 GiB are expert weights, 1.19 GiB is attention, 1.08 GiB is the output projection, 1.08 GiB is the token embedding, and 8.44 MiB are router weights, one router per layer.

During decode, every read of an expert weight from disk is dereferenced inside the matmul that consumes it. Under the default loader, that dereference is what triggers the page fault that brings the slice into RAM.

Across 1999 generated tokens, expert selection is far from uniform. Each MoE layer picks 4 of its 32 experts per token, so a balanced router would select each expert about

250 times over the run. The actual range is 0 to 1635 selections per (layer, expert) pair, and the imbalance is heavily layer-dependent. Layer 10 is concentrated: the median expert is picked only 68 times, well below the 250 of a uniform router, while one expert is picked 1635 times (on 81.79% of tokens). At layer 0 the routing is closer to uniform: the median is 220 and the most-used expert 589.

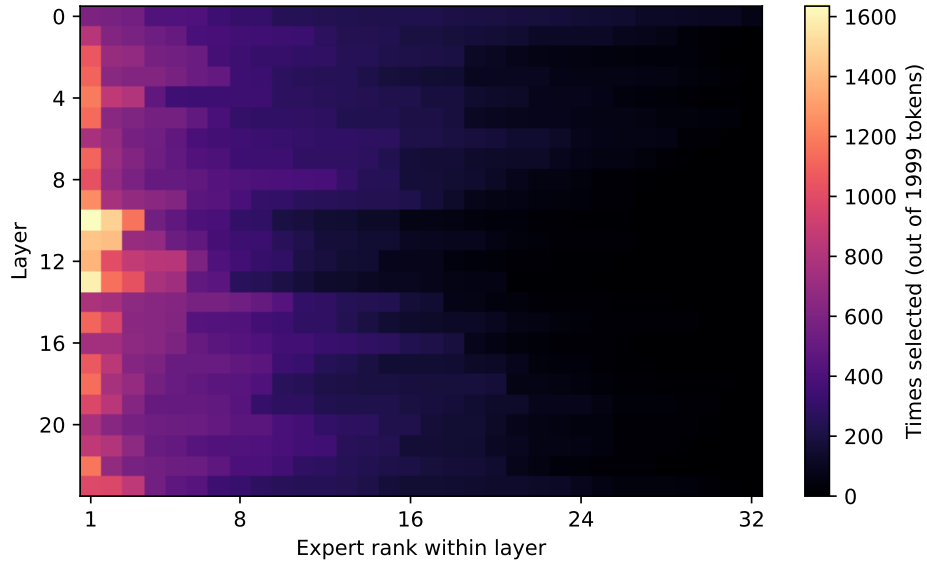


Figure 4.1.: Selection counts for each (layer, expert) pair across 1999 generated tokens. Within each layer (row), the 32 experts are sorted by selection count descending, so column 1 is the layer’s most-selected expert. Layers 10 to 13 fall off steeply (a few experts dominate the rest); layer 0 is much flatter.

The temporal access pattern is sharply skewed toward immediate reuse: 45.31% of (layer, expert) reuses happen at the very next token, 76.14% within five tokens, and 87.12% within ten (Figure 4.2). This is what makes a small in-memory cache effective even when its slot count sits below the per-token working set of 288: the experts that are about to be needed again are usually the ones just used.

4.3. Visualization

Two visualizers consume the per-token JSON. The first is a browser-based viewer for one or two tokens at a time, with three synchronized views: a per-operation timeline, that token’s computation graph, and a heatmap of reads against file offset. The second

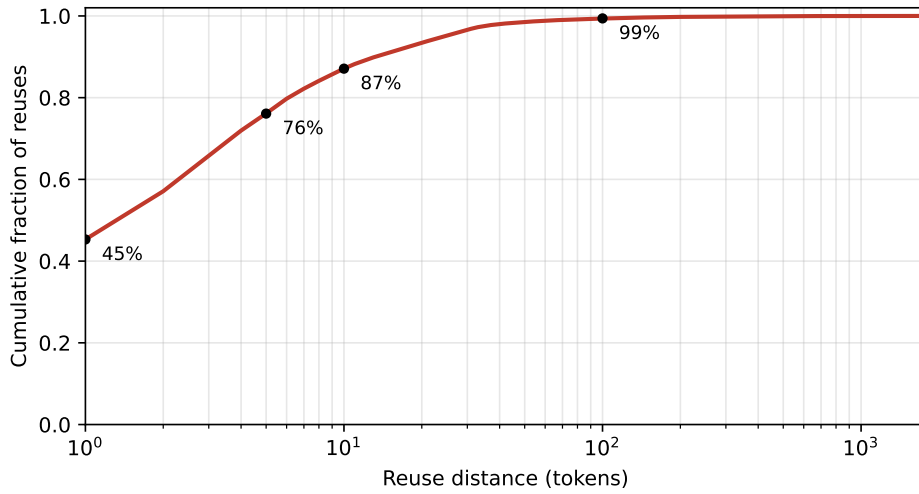


Figure 4.2.: Reuse distance for (layer, expert) selections in the canonical 1999-token decode. Distance is the number of tokens between two consecutive selections of the same (layer, expert) pair.

is a native desktop application that drops the graph view in exchange for handling a hundred tokens or more, used for analysis across many tokens.

4.4. Simulating Replacement

The simulator replays the buffer-access dump under five replacement policies and reports the resulting hit rate at a chosen cache size. We compare LRU, LFU, LFU with periodic counter decay (LFU-aging), Adaptive Replacement Cache (ARC) [MM03], and W-TinyLFU [EFM17].

The cache size relative to the per-token working set of 288 slices determines which policy wins. Below the working set (200 and 250 slots), LRU and ARC produce zero hits: by the time a slice is selected for the second time, LRU has already evicted it, because the recency tail is exactly what is needed next. LFU-aging dominates this regime by holding on to the small set of frequently-routed-to experts. At and above the working set, LRU recovers, because the cache fits a token’s reads with room to spare and per-access recency tracking now captures the immediate reuses of Figure 4.2. LFU’s counters suffer from stale popularity: experts that were heavily selected earlier in the run hold on to their slots after they have cooled. ARC tracks LRU within a percentage point at every cache size where it is non-zero, and W-TinyLFU sits between the two extremes.

Table 4.1.: Cache hit rate by replacement policy and cache size on the canonical 20B trace (1999 tokens, 575,724 expert weight accesses). The per-token working set is 288 slices. Bold marks the best policy at each cache size.

Cache slots	LRU	LFU	LFU-aging	ARC	W-TinyLFU
200	0.00%	13.36%	29.08%	0.00%	25.21%
250	0.00%	16.85%	34.13%	0.00%	31.05%
288	36.02%	19.47%	30.48%	32.92%	33.33%
400	46.31%	28.97%	46.11%	48.04%	42.39%
500	54.88%	33.70%	54.03%	56.32%	50.30%
750	73.32%	46.77%	68.47%	73.68%	70.40%
1000	85.36%	62.02%	79.83%	85.46%	77.46%

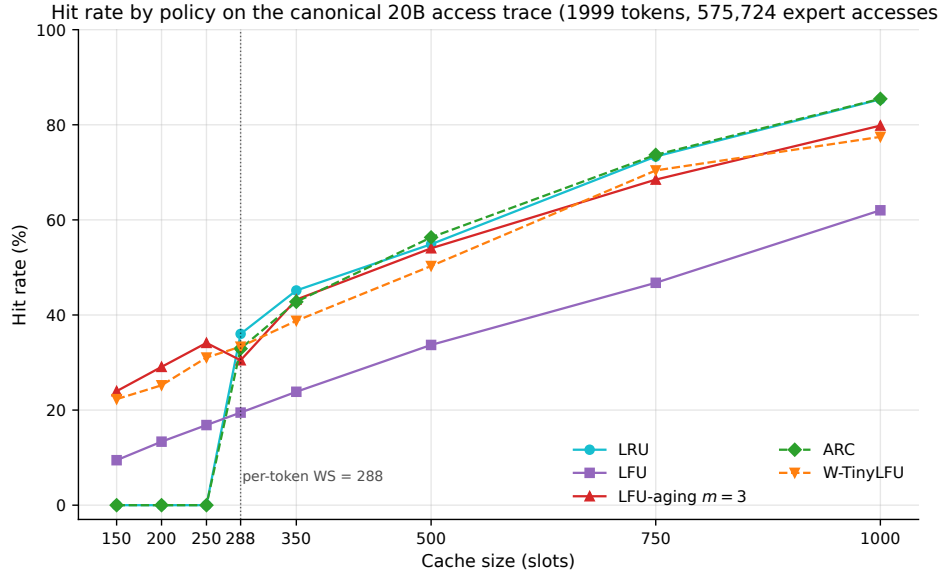


Figure 4.3.: Simulator hit-rate curves on the canonical 20B trace, sweeping cache sizes from 150 to 1000 slots. The dotted vertical line marks the per-token working set of 288 slices, where the regime crossover happens: below it, LRU and ARC drop to zero while LFU-aging dominates; above it, LRU and ARC recover and lead.

The simulator’s LFU-aging implementation reproduces the hit rate of the deployed C runtime to within rounding (34.13% simulated, 33.80% measured at the canonical 7 GiB / 250-slot configuration), so the rankings in Table 4.1 reflect what the runtime would observe. The aging period was tuned on the simulator: counters are halved every $3 \times \text{cache_slots}$ accesses, the highest-hit-rate multiplier we tested. Routing-aware prefetching, which predicts the next layer’s experts so their slices can be read ahead of time, is an active research direction [Jia+25; Xue+24a] and is out of scope here.

5. Implementation

5.1. Pinning the dense weights

The non-expert weights of the model file stay in the kernel-managed mmap region, and we mlock that region so the kernel cannot evict any of them. The mlock excludes the experts (managed by the buffer of Section 5.2) and the input embedding (each generated token reads only one of its rows). Attention, the output projection, the layer norms, and the per-layer routers are therefore guaranteed resident throughout decode.

5.2. The expert buffer

We replace the kernel’s page-fault path for expert weights with `io_uring` [Axb19] reads against an `O_DIRECT` [KL25c] file descriptor (Section 2.4). The application writes read requests into a shared submission ring, the kernel performs them in the background, and `O_DIRECT` keeps the data out of the page cache so it transfers directly from the device into application buffers. Together they let us submit many disk reads at once and pick up completions when convenient, with no intermediate copy in the page cache.

`O_DIRECT` requires every read offset, length, and destination buffer address to be a multiple of 512 bytes. The expert weight tensors in a GGUF file do not satisfy this. We address the alignment with a one-time tool that reads every expert weight slice out of the GGUF and writes it into a sidecar file (`.bscexp`) at a 512-byte-aligned offset. The sidecar is opened once per process with `O_DIRECT`, and every `io_uring` read targets a sidecar offset that is aligned by construction.

The buffer is a single 512-aligned anonymous region holding N slots of 4.20 MiB each, where N is configurable. A hash map keyed by $(layer, projection, expert_id)$ returns the slot index of any resident slice. On a hit, the buffer returns the slot’s address. On a miss, it picks a victim slot under the active replacement policy, issues a single `io_uring` read into that slot, and returns the slot’s address once the completion arrives.

Three replacement policies are implemented: LRU, LFU, and LFU-aging (the policies the simulator selected in Section 4.4). LRU keeps a doubly-linked list and promotes the slot on every access. On a miss, the list’s tail is evicted. LFU and LFU-aging keep

per-slot frequency counters and evict the slot with the smallest counter, with LFU-aging additionally halving every counter once per $3 \times \text{cache_slots}$ accesses to discount stale popularity. A per-load epoch prevents two misses in the same load from picking the same victim slot.

5.3. Connecting the buffer to compute

The buffer's slices live in our own region of memory, separate from the mmap'd region of the model file that GGML's matrix-multiplication code normally reads from. To make the slices visible to compute without copying, we use an existing GGML field: every tensor carries an opaque `extra` pointer that any computation in GGML can read freely. Before each MoE layer runs, we fill `extra` with our array of slot pointers (one per active expert), and we patch the MoE matrix-multiplication function so that when `extra` is set, each active expert's address is taken from the array instead of from the tensor's base. When `extra` is unset, the original code runs and no other operation in the model is affected.

5.4. Pipelining I/O with compute

The simplest mode of the buffer is fully synchronous: a pre-compute callback before each layer submits all 12 reads as one `io_uring` batch and waits for every completion before compute begins. The three modes below progressively overlap I/O with compute, and each produces the same output as the synchronous path at fixed seed.

5.4.1. Projection-group overlap

The down matmul takes the SwiGLU output as input, which means the down weights are not needed until SwiGLU has finished. They can be loaded in the background while up, gate, and SwiGLU run.

We split the buffer's load into two phases inside a single callback before the up matmul. Phase 1 submits the 8 up+gate reads as one `io_uring` batch and waits for every completion. Phase 2 submits the 4 down reads as a separate batch and returns immediately. While the worker threads run up, gate, and SwiGLU on the up+gate slices, the 4 down reads complete in the background. A second callback attached to the down matmul waits for any remaining down completions before down runs.

The 4 experts are still processed in a fixed order: each MoE matmul iterates them as 0,1,2,3 in turn (the positions in which the router emitted them). Only the I/O is split.

The two phases share a single load epoch, which guarantees that phase 1's slots cannot be picked as eviction victims by phase 2.

5.4.2. The fused MoE op

In default llama.cpp, one MoE layer's feed-forward is a chain of separate GGML nodes: matmuls for up, gate, and down, plus SwiGLU and a per-expert accumulation. Each matmul iterates the 4 selected experts internally. The worker pool synchronizes at every node boundary, so all 4 experts finish their up matmul before any starts gate, and so on. The structure is per-operation outside, per-expert inside.

Projection-group overlap fits this layout, because its callbacks attach to the existing nodes. What the layout does not allow is starting one expert's down matmul while another expert is still loading from disk. That would require inverting the loop nesting to per-expert outside, per-operation inside, and the per-node barriers prevent it.

We replace all of those default nodes with one fused custom GGML op. From GGML's perspective, the entire MoE feed-forward is now one node, so GGML imposes no barrier inside it. Inside, thread 0 picks the next compute action and issues any `io_uring` submissions or waits the action requires. All threads cross a barrier and cooperate on the picked action. One barrier per dispatch round is enough, and the granularity at which we interleave I/O and compute is now ours to choose.

5.4.3. Async-projection-overlap

Async-projection-overlap differs from projection-group overlap in two ways. All 12 reads are submitted at the start of the fused op, not split into two phases. The 4 experts are processed in the order their data arrives, not in 0, 1, 2, 3 position order.

Each expert's up and gate reads share one `io_uring` tag and its down read takes a separate tag, for $4 \times 2 = 8$ tags in flight together. Thread 0 waits for any one of the 4 up+gate pairs to complete and dispatches that expert. The worker threads run up, gate, and SwiGLU on that expert. Thread 0 then waits for that expert's down read (in flight since layer entry) before the worker threads run down. Thread 0 picks the next first-ready expert from the remaining tags and the loop repeats until all 4 have run.

Each expert's output is written into a slot at its router-assigned position, and the 4 slots are summed in position order at the end of the op, so the final output matches the synchronous and projection-group overlap paths.

5.4.4. Async-experts

Async-experts pushes the dispatch granularity one step finer. Where async-projection-overlap dispatches per expert (each expert runs up + gate + SwiGLU + down end-to-end

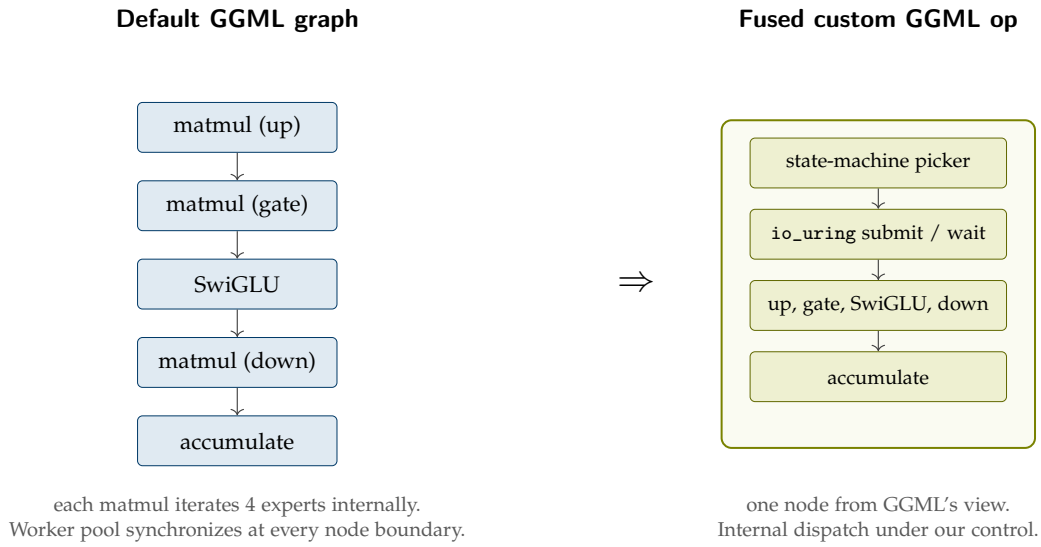


Figure 5.1.: Default GGML graph (left) versus the fused custom op (right). The 5 separate nodes on the left are subject to GGML's per-node barriers, which fix the loop nesting to per-operation outside, per-expert inside. Collapsing them into one node on the right places the dispatch loop inside our control, so the per-expert and per-matmul dispatch in the next two subsections become possible.

before the next expert starts), `async-experts` dispatches per individual matmul: an expert's up matmul can run as soon as its up tag completes, even before its gate read has arrived.

Each projection of each expert gets its own `io_uring` tag, for $4 \times 3 = 12$ tags in flight together. Thread 0 maintains a per-expert state machine and at each dispatch round walks the 4 active experts in order, picking the first eligible action (Figure 5.2). When the second of an expert's two pre-SwiGLU matmuls completes, SwiGLU and the down-input quantization fold into the same dispatch round, so SwiGLU is never a standalone action.

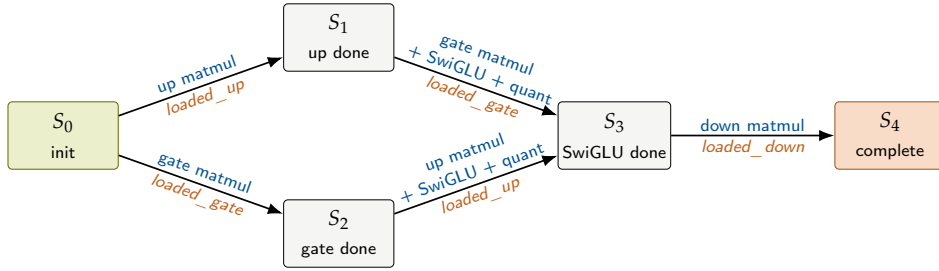


Figure 5.2.: Per-expert state machine inside the `async-experts` dispatch loop. S_0 is the initial state. The diamond $S_0 \rightarrow \{S_1, S_2\} \rightarrow S_3$ captures the difference from `async-projection-overlap`: an expert's first matmul runs as soon as one of its up or gate reads completes, without waiting for the other. The second-arrival matmul folds SwiGLU and the down-input quantization into the same dispatch round (the *and finish* variants), so S_3 is the post-SwiGLU state ready for the down matmul.

The same position-order summation at the end makes the output identical to `async-projection-overlap`. In our wall-clock measurements (Chapter 6), `async-experts` lands within roughly one percent of `async-projection-overlap`, so we keep `async-projection-overlap` as the deployed configuration.

6. Evaluation

This chapter measures our backend against the default llama.cpp loader on the same hardware under a kernel-enforced RAM limit.

6.1. Setup

The test machine runs Linux on an AMD Ryzen 7 7700X (8 cores) with 30 GiB of DDR5 main memory and two NVMe SSDs [Adv22; JED20]. The operating system lives on the first SSD. The second is a 1 TB Samsung 980 PRO over PCIe 4.0 [PCI17] dedicated to the model files and their sidecar pack files (Section 5.2); all reported numbers come from this drive, so model I/O is not mixed with operating-system activity during measurement.

We control RAM by launching the inference process inside a transient systemd scope under cgroup v2 with `memory.max` set to the limit we want and `MemorySwapMax=0`, with system swap also disabled [Heo15]. An allocation that would push the process over the cgroup limit fails immediately rather than spilling to disk. Each measurement runs 2000 tokens of decode at seed 42 from a multi-topic prompt, kernel page caches are dropped before every run, and the reported throughput is decode token count divided by decode time. All measurements use the `-eager-compute` flag, which forces the compute buffer resident at graph reservation so the cgroup count is deterministic.

The cgroup limit covers everything the inference process holds. A fixed footprint goes to the pinned dense weights (Section 5.1), the KV cache, the compute buffer, and a small amount of miscellaneous anonymous memory. The remainder, less a safety margin, holds our expert buffer (Section 5.2) at 4.20 MiB per slot. Table 6.1 lists the breakdown for both models.

For each cgroup limit, the cache slot count is sized to fill the remaining space: 250 / 498 / 740 slots at 7 / 8 / 9 GiB on GPT-OSS-20B, and 693 / 1668 / 3130 / 4592 slots at 12 / 16 / 22 / 28 GiB on GPT-OSS-120B. Across the 168 runs of the canonical sweep (56 cells $\times$ 3 iterations across both models), the mean coefficient of variation in decode time is 0.23%.

Table 6.1.: Fixed memory footprint of the inference process under the cgroup limit. The remainder of the limit, less a safety margin, holds the expert buffer at 4.20 MiB per slot.

Component	GPT-OSS-20B	GPT-OSS-120B
Pinned dense weights	2.27 GiB	2.98 GiB
KV cache (ctx=131072)	3.02 GiB	4.53 GiB
Compute buffer (eager)	0.40 GiB	0.40 GiB
Misc anonymous	0.18 GiB	0.23 GiB
Fixed footprint (sum)	5.87 GiB	8.14 GiB

6.2. GPT-OSS-20B

Table 6.2 reports decode throughput on GPT-OSS-20B (12.85 GiB on disk) at four cgroup limits. At the smallest three cells (7, 8, 9 GiB), the working set exceeds the available RAM once the pinned weights, KV cache, and compute buffer are accounted for, so the model file’s pages have to turn over every layer. The 22 GiB cell leaves enough RAM for the model to stay fully resident.

The configuration order is monotonic at the three smaller cells. Pinning the dense weights (Section 5.1) helps in proportion to what the kernel would otherwise evict: $1.55\times$ over lazy mmap at 7 GiB, $1.21\times$ at 9 GiB. Replacing the synchronous fault path with our buffer plus projection-group overlap (Subsection 5.4.1) adds the largest single jump, roughly a further factor of two over pin. Async-projection-overlap (Subsection 5.4.3) reaches 5.36 / 6.52 / 8.70 tok/s at 7 / 8 / 9 GiB ($3.72\times$, $2.44\times$, and $2.25\times$ over the lazy mmap baseline) and is the fastest configuration at 7 and 9 GiB. At 8 GiB async-experts (Subsection 5.4.4) edges it at 6.59 tok/s, by an amount that sits inside the run-to-run noise of that cell (Section 6.5).

The 22 GiB cell is the upper bound on this hardware for this model. With the working set entirely resident, the kernel page cache holds every weight, the synchronous fault path stops being a cost we have to remove, and all five configurations converge to within 0.42 tok/s of each other at 13.90–14.32 tok/s. Our backend in fact loses about 0.40 tok/s relative to plain mmap at this cell. The cache is sized to 2304 slots, exactly the full expert population (24 layers $\times$ 32 experts $\times$ 3 projections), so once it warms neither path makes runtime I/O and what remains is overhead in the buffer-managed code path that the I/O savings at smaller cells more than offset. Every speedup the rest of this chapter reports is the speedup available when the RAM cap is tight. In its absence the buffer-managed I/O path is a small net cost.

Table 6.2.: GPT-OSS-20B throughput at four cgroup limits, three iterations per cell. Each block is monotonic top to bottom when the model does not fit in RAM: pinning helps in proportion to what the kernel would otherwise evict, and our buffer-managed I/O stack adds roughly an additional $2\times$ on top. At 22 GiB the working set fits and all configurations converge.

Configuration	Tok/s	Eval CV (%)	Speedup vs lazy
<i>cgroup memory.max = 7 GiB</i>			
lazy mmap	1.44	0.15	1.00×
mmap + pin	2.24	0.76	1.55×
<i>with buffer: 250 slots (≈ 1.03 GiB), ≈ 0.10 GiB free</i>			
projection-group overlap	5.02	0.32	3.49×
async-projection-overlap	5.36	0.48	3.72×
async-experts	5.11	1.63	3.55×
<i>cgroup memory.max = 8 GiB</i>			
lazy mmap	2.67	0.04	1.00×
mmap + pin	3.50	0.17	1.31×
<i>with buffer: 498 slots (≈ 2.04 GiB), ≈ 0.08 GiB free</i>			
projection-group overlap	6.18	0.09	2.31×
async-projection-overlap	6.52	0.18	2.44×
async-experts	6.59	2.29	2.47×
<i>cgroup memory.max = 9 GiB</i>			
lazy mmap	3.86	0.08	1.00×
mmap + pin	4.68	0.12	1.21×
<i>with buffer: 740 slots (≈ 3.04 GiB), ≈ 0.09 GiB free</i>			
projection-group overlap	8.14	0.01	2.11×
async-projection-overlap	8.70	0.03	2.25×
async-experts	8.57	0.01	2.22×
<i>cgroup memory.max = 22 GiB; model fits in RAM</i>			
lazy mmap	14.32	0.04	1.00×
mmap + pin	14.29	0.06	1.00×
projection-group overlap	14.04	0.06	0.98×
async-projection-overlap	13.92	0.02	0.97×
async-experts	13.90	0.11	0.97×

Table 6.3.: GPT-OSS-120B throughput at four cgroup limits, three iterations per cell. GPT-OSS-120B (60.88 GiB on disk) exceeds the host’s 30 GiB RAM by a factor of about 2, so every cell forces page-cache turnover regardless of the cgroup limit. The configuration ranking is the same as on the smaller model, and our buffer-managed I/O stack adds a factor of roughly 1.63–2.38 $\times$ on top of pin.

Configuration	Tok/s	Eval CV (%)	Speedup vs lazy
<i>cgroup memory.max = 12 GiB</i>			
lazy mmap	1.50	0.10	1.00 $\times$
mmap + pin	1.89	0.21	1.26 $\times$
<i>with buffer: 693 slots (≈ 2.84 GiB), ≈ 1.02 GiB free</i>			
projection-group overlap	3.37	0.09	2.25 $\times$
async-projection-overlap	3.58	0.02	2.38 $\times$
async-experts	3.54	0.03	2.36 $\times$
<i>cgroup memory.max = 16 GiB</i>			
lazy mmap	2.25	0.15	1.00 $\times$
mmap + pin	2.55	0.16	1.13 $\times$
<i>with buffer: 1668 slots (≈ 6.84 GiB), ≈ 1.02 GiB free</i>			
projection-group overlap	4.55	0.23	2.02 $\times$
async-projection-overlap	4.89	0.02	2.17 $\times$
async-experts	4.81	0.16	2.13 $\times$
<i>cgroup memory.max = 22 GiB</i>			
lazy mmap	3.50	0.12	1.00 $\times$
mmap + pin	3.72	0.10	1.07 $\times$
<i>with buffer: 3130 slots (≈ 12.84 GiB), ≈ 1.03 GiB free</i>			
projection-group overlap	6.23	0.25	1.78 $\times$
async-projection-overlap	6.66	0.02	1.90 $\times$
async-experts	6.57	0.05	1.88 $\times$
<i>cgroup memory.max = 28 GiB</i>			
lazy mmap	4.96	0.33	1.00 $\times$
mmap + pin	5.17	0.18	1.04 $\times$
<i>with buffer: 4592 slots (≈ 18.83 GiB), ≈ 1.03 GiB free</i>			
projection-group overlap	7.70	0.30	1.55 $\times$
async-projection-overlap	8.06	0.04	1.63 $\times$
async-experts	7.99	0.01	1.61 $\times$

Table 6.4.: Cache eviction policy comparison at GPT-OSS-20B, cgroup memory.max = 7 GiB, cache 250 slots (below the per-token working set of 288). Three iterations per cell, all CVs $\leq 1.7\%$. All values in tokens per second. LFU-aging dominates LFU dominates LRU at every io_uring code path: when the cache is below the working set, the LRU tail is exactly the next slice the access pattern asks for and the policy thrashes.

io_uring code path	LRU	LFU	LFU-aging $m = 3$
uring (sync)	3.65	4.15	4.82
projection-group overlap	3.80	4.31	5.02
async-projection-overlap	3.98	4.58	5.36
async-experts	3.87	4.38	5.11

Table 6.5.: Reproducibility of the May 1/2 canonical sweep across all 56 configurations and 168 total iterations. Mean coefficient of variation in eval seconds is 0.23%; median 0.101%; max 2.29%. The kernel-enforced cgroup memory.max methodology is reproducible at the sub-percent level on this hardware.

Statistic	Value
Configurations	56
Iterations / cell	3
Total runs	168
Mean CV (%)	0.23
Median CV (%)	0.101
Max CV (%)	2.29
Configs with CV < 0.5%	49 of 56

6.3. GPT-OSS-120B

Table 6.3 reports the same sweep on GPT-OSS-120B (60.88 GiB on disk). The model exceeds host RAM by a factor of about 2, so every cell forces page-cache turnover regardless of the cgroup limit; even at 28 GiB the page cache cannot hold the full set of expert weights.

The configuration order is the same as on the smaller model. Pinning helps less in absolute terms (4–26% over lazy mmap rather than 21–55%) because the cgroup budgets here (12–28 GiB) leave the kernel page cache more headroom than the 2.98 GiB of dense weights need: even the 12 GiB cell has roughly 6.8 GiB free for the page cache, so the kernel’s own LRU keeps most dense pages resident without help. Pinning’s marginal value is what the kernel would otherwise evict, and at these budgets that is a small share of dense reads. Async-projection-overlap is the fastest configuration at every cell, reaching 8.06 tok/s at 28 GiB against 4.96 tok/s for lazy mmap, a $1.63\times$ speedup. There is no fully-resident cell here in our cgroup range: even at 28 GiB the model is more than twice the available RAM, so I/O dominates every run.

6.4. Cache eviction policy at undersized cache

The 7 GiB cell on GPT-OSS-20B fixes the buffer at 250 slots, which is below the 288-slice per-token working set established in Section 4.4. The simulator there predicts that LFU-aging dominates LRU when the cache holds fewer slots than the working set. Table 6.4 confirms this at wall-clock time across all four `io_uring` code paths.

At the headline `async-projection-overlap` cell, LFU-aging reaches 5.36 tok/s against LRU’s 3.98 tok/s. The simulator’s hit-rate prediction in Table 4.1 explains the gap: LRU at 250 slots produces a 0.00% hit rate, so every expert access pays a fresh disk read, while LFU-aging holds 34.13%. The mechanism behind the zero (described in Section 4.4) is that the cyclic per-token access pattern places LRU’s tail on exactly the slice it is about to ask for next, so each new arrival evicts what is needed immediately afterward. Per-slot frequency counters protect the small set of frequently-routed-to experts and avoid that geometry. At 8 GiB and 9 GiB the cache (498 / 740 slots) exceeds the working set and we use LRU, because each expert’s three slices (up, gate, down) are read in immediate succession within a layer, which LRU’s per-access promotion captures cleanly. The same regime argument applies to every GPT-OSS-120B cell in our sweep, where the cache always holds more slots than the per-token working set.

6.5. Comparing the overlap modes

The three pipelining modes that overlap I/O with compute (projection-group overlap, async-projection-overlap, async-experts) all produce the same output at fixed seed, so the comparison between them is purely wall-clock.

Across the seven cells where the model does not fit (GPT-OSS-20B at 7, 8, 9 GiB; GPT-OSS-120B at 12, 16, 22, 28 GiB), async-projection-overlap is the fastest at six of seven. Projection-group overlap consistently lags it by 4.47 to 6.95% (on GPT-OSS-20B at 9 GiB it is 8.14 vs 8.70 tok/s, a 6.44% gap). Async-experts lands within 1.49% of async-projection-overlap at every cell, for example 8.57 vs 8.70 tok/s at GPT-OSS-20B 9 GiB and 7.99 vs 8.06 tok/s at GPT-OSS-120B 28 GiB. The 20B 8 GiB cell is the only one where async-experts edges async-projection-overlap (6.59 vs 6.52 tok/s), by an amount inside the cell’s run-to-run noise.

Async-projection-overlap is therefore the deployed configuration. Async-experts is within noise of it on this hardware. Projection-group overlap is a few percent behind because the down reads only have one compute window to hide behind, while the two async modes can overlap each expert’s I/O with the compute of every preceding expert in the layer.

7. Discussion and Conclusion

7.1. Limitations

The headline numbers come from a single host running a multi-topic prompt at a fixed seed across both models. Three caveats follow from that.

- The hardware is one consumer-class configuration: an 8-core AMD CPU, 30 GiB of DDR5, and a single PCIe 4.0 NVMe SSD as the model store. A different CPU would shift the 22 GiB CPU-bound ceiling on GPT-OSS-20B. A slower SSD would shrink the absolute speedups of the buffered modes.
- Wall-clock numbers come from one prompt domain. We separately captured tensor-level traces on five additional prompts (code, mathematical reasoning, creative writing, factual explanation, and a mixed code-plus-prose prompt). The simulator’s hit rate at the headline 250-slot LFU-aging configuration spans 32.6% (mixed) to 43.0% (creative), a 10.4 pp spread, and reuse-distance-1 spans 43–50% across the same six domains. Translating the hit-rate spread through our empirical hit-rate-to-tok/s sensitivity points to a few percent of cross-domain wall-clock variation, but we did not run end-to-end wall-clock measurements on the five extra domains.
- Two replacement policies (ARC and W-TinyLFU) appear only in the simulator, not in the C runtime. The simulator’s predictions match the deployed policies to within rounding, but a live C implementation would close the loop in the operating regime where LRU and LFU-aging already perform within noise of each other.

7.2. Future work

Four directions extend naturally from where this thesis stops.

Cross-layer expert prefetching. The pipelining work in this thesis stops at the layer boundary, because the router’s choice of next-layer experts is independent of the current

layer’s hidden state in any way we can predict cheaply. A small router-prediction head, trained to forecast layer $N+1$ ’s expert IDs from layer N ’s output, would let the loader start the disk reads one layer earlier and hide more of the I/O behind compute. This is a separate research direction (expert speculation).

Request-similarity prefetch. The current system carries no state between requests. Every request starts with an empty cache and pays the cold-start miss for each first-touch expert. MoE-Infinity [Xue+24a] records each completed request as a small Expert Activation Matrix (rows are layers, columns are experts, entries are how often each expert was selected) and matches an in-flight request against historical ones by cosine similarity to drive prefetch. The construction transfers cleanly to our setting: a few kilobytes per stored matrix, microsecond cosine matching, and a populated cache before the first decode token. It targets a phase the per-token working-set arithmetic in Chapter 4 cannot address, and it is orthogonal to the router-prediction head above (cross-request fingerprint at request start versus cross-layer router prediction during decode).

Faster startup via lazy pinning. The current pinning mlocks the dense weight pages eagerly at model-load time. Linux’s `mlock2` with the `MLOCK_ONFAULT` flag [KL25a] marks pages non-reclaimable but defers the actual residency until first fault. For a generation-time workload this would shift the load-time cost into the first forward pass without changing steady-state behaviour.

GPU integration. The work in this thesis is entirely CPU-side. The I/O architecture (the buffer manager, the sidecar pack file, the per-expert tagged completions) is orthogonal to the compute backend. A GPU compute path would still need expert weights resident in some buffer before launch and the same routing-driven cache. The integration question is the boundary: `0_DIRECT` reads can target GPU-resident memory through GPUDirect Storage [NVI26] or analogous facilities, pushing the buffer onto the device and keeping the CPU out of the data path.

7.3. Conclusion

Modern open-source language models hold large amounts of weight data on disk. The two studied here, GPT-OSS-20B and GPT-OSS-120B, weigh 12.85 GiB and 60.88 GiB respectively, against the 30 GiB of RAM on our test machine. RAM is expensive, NVMe SSDs are not, and a Mixture-of-Experts model reads only a small fraction of its weights per generated token.

We built a tensor-level tracer to see what the model actually reads, and the trace showed a non-uniform access pattern: the dense per-layer weights are read on every token, while only a few experts per layer per token are selected by the router. We pinned the dense weights in RAM so the kernel could not evict them, and we managed the experts ourselves in a buffer that reads from the SSD asynchronously, with three loading strategies of increasing aggressiveness. The most aggressive submits all of a layer’s expert reads at the start of the layer and dispatches each expert’s compute as its data arrives.

With 7 GiB of RAM available, our backend lifts GPT-OSS-20B from 1.44 tok/s under the default loader to 5.36 tok/s, almost four times faster. GPT-OSS-120B does not fit in our 30 GiB of RAM at all. The default loader runs it at 4.96 tok/s and our backend at 8.06 tok/s. A simulator built on the same trace lays a foundation for choosing the cache replacement policy and confirms the wall-clock observation that LFU-aging dominates when the cache is below the per-token working set, while LRU wins above it.

The tracer, the buffer-managed loader, the simulator, and the patched llama.cpp fork are released as artifacts (Appendix A) so the path from question to answer can be walked again on different hardware, a different model, or a different storage stack.

A. Software Artifacts

Prerequisites. The reproduction recipes below assume a Linux 5.6+ kernel with cgroup v2 [Heo15] mounted at `/sys/fs/cgroup/`, liburing 2.0+ headers, GCC 11+, CMake 3.20+, and a user account with `sudo` for cgroup creation and kernel-cache drop operations. System swap must be disabled (`sudo swapoff -a`) so cgroup `MemorySwapMax=0` is enforceable end-to-end. The model file must be a GPT-OSS-20B or GPT-OSS-120B GGUF in F16 quantization [Ope25] from the `ggml-org/gpt-oss-*-GGUF` repositories on Hugging Face. The `.bscexp` sidecar is produced by the `bsc-pack-experts` preprocessor at first run (Section A.1). The test machine described in Section 6.1 is the reference hardware; the methodology transfers to any host with NVMe-attached storage.

A.1. Forked llama.cpp

Repository: `github.com/ErsjanKeri/llama.cpp`, branch `expert-pipeline-fused`, head `d7e480f4`. The fork carries twenty-four commits on top of upstream `llama.cpp` [Gl23] plus a working-tree diff covering the `async-projection-overlap` and `async-experts` pipelines (sixteen tracked source files in the `llama-io-uring`, `llama-moe-pipeline`, and `llama-graph` translation units, the `arg` and `common` units in `common/`, and a small change in `include/llama.h`). The thesis state is HEAD `d7e480f4` plus that working-tree diff; line numbers cited below describe the working tree.

The three pipeline-selection flags are mutually exclusive; the constraint is enforced in `common/arg.cpp`.

Key code paths added.

- `src/llama-io-uring.{h,cpp}`, `src/llama-io-uring-buf.{h,cpp}`: application-managed expert loader and cache buffer.
- `src/llama-moe-pipeline.{h,cpp}`: fused MoE FFN custom op for the overlap variants.
- `src/llama-graph.cpp`: graph integration, the phase callbacks, the `make_compact` view fix, and the `tensor->extra` wiring.

Table A.1.: Runtime flags introduced by the fork.

Flag	Purpose
-pin-compute-weights	mlock attention, output, and norm weights (excludes experts and embedding)
-eager-compute	memset compute buffer at graph_reserve so its 413 MiB is physically resident
-uring-experts	enable the application-managed expert loader (requires .bscexp)
-uring-cache-slots N	cache size in slots; 0 disables the cache
-uring-cache-policy	eviction policy: lru, lfu, or lfu-aging
-uring-aging-mult N	LFU-aging period multiplier (default 10, tuned optimum 3)
-uring-projection-overlap	projection-group overlap (down async, up+gate sync)
-uring-async-projection-overlap	async-projection-overlap (split-tag, first-ready dispatch)
-uring-async-experts	async-experts (per-(expert, projection) tags, eager dispatch)
-trace-mode	tensor trace filter: off, experts, or all

- ggml/src/ggml-cpu/ggml-cpu.c:1630-1644: patched per-expert pointer indirection in mul_mat_id.
- ggml/include/tensor_trace.h, ggml/src/tensor_trace.c: tensor tracer.
- tools/bsc-pack-experts/: offline .bscexp pack-file builder.
- tools/gguf-dump/gguf-dump.cpp: MXFP4-aware metadata exporter.

A.2. Tensor-Tracing Toolchain

Repository directory: `~/BSC/tensor-tracing/`. The toolchain consumes the binary trace emitted by `ggml/src/tensor_trace.c` (1024-byte cache-aligned records, see Chapter 4) plus the GGUF metadata CSV from `llama-gguf-dump` and the per-token Graphviz `.dot` files from `llama.cpp`, and produces JSON payloads for two visualization frontends.

Visualization frontends. Two were built because no single technology stack handles both interactive 1–2 token introspection and 100+ token bulk analysis well.

The WebUI lives at `webui/`. Stack: React 19, Zustand 5 for state, Three.js 0.182 with `@react-three/fiber/drei` for the 3D heatmap, Cytoscape.js 3.33 [Fra+16] with

Table A.2.: Python parsers in tools/ of the tensor-tracing toolchain.

File	Purpose
parse_trace.py	1024-byte binary trace records from /tmp/tensor_trace.bin into per-token JSON; mirrors the C ggml_op enum at lines 53–220
parse_csv.py	GGUF metadata CSV into memory-map.json; emits per-expert rows for the 2,691-tensor GPT-OSS-20B map
parse_dot.py	per-token Graphviz .dot graphs into JSON nodes/edges, with layer-ID extraction for both blk.N. and name-N suffix patterns
parse_buffer_stats.py	/tmp/buffer_stats.jsonl into an alloc/dealloc timeline plus per-buffer summary (ANY / WEIGHTS / COMPUTE)
preprocess_all.py	orchestrator that runs the four parsers above and lays out webui/public/data/
simulate_from_dump.py	policy simulator: replays the access sequence dumped via BSC_CACHE_DUMP against LRU, LFU, LFU-aging, ARC, W-TinyLFU, and Belady. LRU and LFU-aging match the deployed C runtime to within rounding (34.13% simulated vs 33.80% measured at 7 GiB, c=250).

cytoscape-dagre for graph layout, react-window for virtual scrolling, Vite 7, Tailwind-CSS 3.4, TypeScript strict mode. It has four primary views (GraphView, TraceView, HeatmapView, ViewContainer) and is well-suited to 1–2 tokens (around 1,700 trace entries); beyond ten tokens, React reconciliation cost dominates.

The DesktopUI lives at `desktopui/`. Stack: C++17, OpenGL, GLFW3, vendored ImGui and nlohmann/json. `TraceTableView` reaches 110 FPS at 170,000+ rows via virtual scrolling, and `launch_all_domains.sh` opens one window per semantic domain. This is the workhorse for 100+ token analyses.

Both frontends consume identical JSON schemas, so a trace produced for one is loadable by the other.

A.3. Experiment Harness

Repository directory: `~/BSC/time-tracking/`. The canonical runner is `run_cgroup_experiments.py`, which executes each iteration of llama-completion inside a fresh `bscexp.slice` cgroup-v2 scope created by `systemd-run`. The slice name is deliberately hyphen-free, because `bsc-experiment.slice` would land in `/sys/fs/cgroup/bsc.slice/bsc-experiment`; `systemd` treats hyphens as cgroup-hierarchy separators, and the diagnostic stats reader expects flat paths under `/sys/fs/cgroup/bscexp.slice/`.

A bash wrapper inside the scope reads `memory.peak`, `memory.current`, `memory.swap.peak`, and `memory.events` from the still-existing cgroup before `systemd` tears the scope down, emitting `BSC_CGROUP_*=...` lines for the parser to scrape. Each iteration is preceded by `systemctl stop bscexp.slice`, `drop_caches`, and a 5-second settle. All monitoring runs outside the cgroup so it never competes for the budget.

Configuration files. Per-experiment settings live in JSON files alongside the harness. The two that drive the canonical sweep are `settings_thesis_final.json` (GPT-OSS-20B) and `settings_thesis_final_120b_retry.json` (GPT-OSS-120B).

Result archive layout. Output lands in `results/cgroup_<timestamp>/`, with one CSV per configuration plus the captured stdout of every iteration as `.log` files. Because `systemd-run` executes the inner command as root for cgroup setup, the result directories are root-owned; subsequent reads from user processes need either `sudo` or a follow-up `chown`. The harness does not `chown` automatically because doing so would race with downstream parsers in some setups.

A.4. Datasets and Result Archives

The thesis cites four classes of primary artifact. All paths are absolute under `~/BSC/`.

Research journal. `journal/INDEX.md` is the annotated index, listing every dated entry with its status (`AUTHORITATIVE`, `REFERENCE ONLY`, `DEPRECATED-NUMBERS`). Daily entries live as `journal/2026-MM-DD.md`; multi-day work folds into a dated directory with `summary.md`.

Cited result directories. `time-tracking/results/` holds CSV output for the canonical sweep. The authoritative directories are `cgroup_20260501_191023/` (GPT-OSS-20B, 96 runs, 32 configurations $\times$ 3 iterations including the LRU/LFU/LFU-aging policy sweep at the tight 7 GiB cell) and `cgroup_20260502_140144/` (GPT-OSS-120B retry, 72 runs, 24 configurations $\times$ 3 iterations across the four cgroup budgets, LRU policy throughout because cache exceeds the per-token working set at every budget). The driving settings are `settings_thesis_final.json` and `settings_thesis_final_120b_retry.json`. The corresponding deterministic generation outputs at the canonical seed=42 are stored as `generation_outputs/{20b,120b}_multitopic_seed42.txt`.

A.5. Reproducing the Headline Numbers

The canonical sweep at `cgroup_20260501_191023/` (GPT-OSS-20B) and `cgroup_20260502_140144/` (GPT-OSS-120B) produced the headline numbers cited in Chapter 6. Reproducing them takes five steps on the same hardware, with the same kernel and the same NVMe.

Step 1 clones the fork and checks out the canonical commit. The thesis state is HEAD `d7e480f4` plus the working-tree diff documented in Section A.1; if a tagged thesis commit has been produced by the time of reproduction, check out that tag instead.

```
# 1. Clone the fork and check out the canonical commit.
git clone https://github.com/ErsjanKeri/llama.cpp ~/llama.cpp
cd ~/llama.cpp && git checkout d7e480f4
# Apply the working-tree diff if reproducing the dirty state:
# git apply path/to/working-tree.diff

# 2. Build llama-completion and bsc-pack-experts.
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --target llama-completion bsc-pack-experts -j$(nproc)

# 3. Produce the .bscexp sidecar pack file for the model.
```

```
./build/bin/bsc-pack-experts <model.gguf> -o <model.gguf>.bscexp --verify
```

```
# 4. Switch to the harness directory.
```

```
cd ~/BSC/time-tracking
```

```
# 5a. Run the canonical 20B sweep.
```

```
sudo python3 run_cgroup_experiments.py \  
--settings settings_thesis_final.json
```

```
# 5b. Run the canonical 120B retry sweep.
```

```
sudo python3 run_cgroup_experiments.py \  
--settings settings_thesis_final_120b_retry.json
```

The harness drops the page cache between iterations, wraps each llama-completion invocation in a fresh `bscexp.slice` cgroup with `MemorySwapMax=0`, runs three iterations per configuration, and writes one CSV per configuration into `results/cgroup_<timestamp>/.`

The CSVs should match the canonical numbers within the cited CV bounds: mean CV across the 168-run sweep is 0.23%, with the noisiest cell at 2.29% (20B 8 GiB async-experts). If they do not, the most likely cause is a working-tree drift in the fork: re-check the commit hash and the working-tree diff against Section A.1. The next most likely cause is page-cache contamination between iterations on a host where the harness was not run with `sudo`; `drop_caches` requires root.

Abbreviations

List of Figures

1.1. Decode throughput of the default llama.cpp loader against our backend	2
2.1. Decoder-only transformer block	4
2.2. Mixture-of-Experts layer for one generated token	5
2.3. Read flow under io_uring with O_DIRECT	7
2.4. Eviction under io_uring with O_DIRECT	7
4.1. Per-layer expert selection counts	13
4.2. Reuse-distance CDF for (layer, expert) selections	14
4.3. Simulator hit rate by policy across cache sizes	15
5.1. Default GGML graph versus the fused MoE op	20
5.2. Per-expert state machine for async-experts	21

List of Tables

4.1. Cache hit rate by policy and cache size	15
6.1. Fixed memory footprint per model	23
6.2. GPT-OSS-20B throughput at four cgroup budgets	24
6.3. GPT-OSS-120B throughput at four cgroup budgets	25
6.4. Cache eviction policy at 20B 7 GiB c=250	26
6.5. Reproducibility of the May 1/2 canonical sweep	26
A.1. Runtime flags introduced by the fork.	33
A.2. Python parsers in tools/ of the tensor-tracing toolchain.	34

Bibliography

- [Adv22] Advanced Micro Devices. *AMD Ryzen 7 7700X Desktop Processor*. Verified 2026-05-05 (corroborated through AMD’s product page, TechPowerUp, and Wikipedia after the AMD page itself failed to respond within timeout). Zen 4 architecture, 8 cores / 16 SMT threads, 4.5 GHz base / 5.4 GHz boost, 32 MB L3 cache, socket AM5, released 27 September 2022. Cited in §6.1 for the 8-core count of the test machine’s CPU. Sept. 2022.
- [Ali+24] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar. “LLM in a flash: Efficient Large Language Model Inference with Limited Memory.” In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Verified 2026-05-05 against arXiv 2312.11514 (v1: 12 Dec 2023; latest v3: 30 Jul 2024). Authors and ACL 2024 venue confirmed. The abstract directly supports the claims we cite in §3.1: storing parameters on flash and paging on demand, the “windowing” reuse strategy, “row-column bundling” for sequential transfers, and the reported $4\text{--}5\times$ CPU and $20\text{--}25\times$ GPU speedups against naive loading. The 90% ReLU FFN post-activation sparsity figure cited in the same paragraph is in the body of the paper, not its abstract. 2024. arXiv: 2312.11514 [cs.LG].
- [Axb19] J. Axboe. *Efficient IO with io_uring*. Technical article, https://kernel.dk/io_uring.pdf. Verified 2026-05-05. Self-published technical article by io_uring’s primary author and Linux block-layer maintainer, hosted on his personal site kernel.dk; PDF metadata dates the document to October 2019. Covers the design rationale (§3), the SQ/CQ shared-ring architecture (§4, “Enter io_uring”), and the io_uring_sqe / io_uring_cqe data structures (§4.1) that we summarise in §2.4 of the thesis. Not the Kernel Recipes 2019 talk (which is a separate artefact titled “Faster IO through io_uring”). Oct. 2019.
- [EFM17] G. Einziger, R. Friedman, and B. Manes. “TinyLFU: A Highly Efficient Cache Admission Policy.” In: *ACM Transactions on Storage* 13.4 (2017). Verified 2026-05-05 against the ACM Digital Library and the dblp record. Previous bib entry used @inproceedings with booktitle = ACM TOS,

which is structurally wrong because ACM TOS is a journal. Corrected to @article. The arXiv version (1512.00727) is from 2015. Cited in §4.4 (tracer simulator chapter) for the W-TinyLFU policy., 35:1–35:31. arXiv: 1512.00727 [cs.OS].

- [Fra+16] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, and G. D. Bader. “Cytoscape.js: a graph theory library for visualisation and analysis.” In: *Bioinformatics* 32.2 (2016). Verified 2026-05-05 against the Oxford Bioinformatics article page. Authors, volume, issue, pages, and DOI confirmed. Cited in Section A.2 for the JavaScript graph-layout library used by the WebUI’s tracer-graph view., pp. 309–311. doi: 10.1093/bioinformatics/btv557.
- [FZS22] W. Fedus, B. Zoph, and N. Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.” In: *Journal of Machine Learning Research* 23.120 (2022). Verified 2026-05-05 against arXiv 2101.03961 (v1: 11 Jan 2021). Authors confirmed. Published in JMLR vol. 23. Switch Transformers simplify MoE routing to top-1 (a special case of top-k); we cite the work in the §3 positioning paragraph alongside [Sha+17] as the canonical line of MoE-with-top-k routing that GPT-OSS inherits from., pp. 1–39. arXiv: 2101.03961 [cs.LG].
- [Gg23] G. Gerganov and ggml contributors. *GGUF File Format Specification*. Verified 2026-05-05. Canonical GGUF specification, maintained in the ggml repository (moved from ggerganov/ggml to ggml-org/ggml; the original URL redirects). Document defines gguf_header_t and gguf_tensor_info_t structures whose fields are name, dimensions, type, and offset; the spec notes that the per-tensor offset is relative to the start of the data section, not the start of the file. 2023.
- [Gl23] G. Gerganov and llama.cpp contributors. *llama.cpp: LLM inference in C/C++*. Verified 2026-05-05. Open-source project started by Georgi Gerganov in March 2023; the canonical repository has since moved from github.com/ggerganov/llama.cpp to github.com/ggml-org/llama.cpp (the original URL redirects). MIT licensed. Cited for the upstream engine that our fork is built on, the GGUF model file format, and the ggml graph executor. Mar. 2023.
- [Heo15] T. Heo. *Control Group v2*. The Linux Kernel documentation. Verified 2026-05-05. Authoritative documentation by the cgroup v2 author Tejun Heo, hosted at docs.kernel.org. Originally published with the cgroup v2 merge into the Linux kernel in 2015 (release 4.5) and continuously updated since. Cited for the memory.max interface (a hard limit that invokes the OOM

killer when reached) and the `memory.swap.max` control we use together to enforce a reproducible RAM cap. 2015.

- [Int25] IntuitionLabs. *RAM Shortage 2025: How AI Demand is Raising DRAM Prices*. Verified 2026-05-05: IntuitionLabs, citing Counterpoint Research. Reports a roughly 50% DDR5 contract-price rise in 2025 plus a further roughly 50% forecast through early 2026, compounding to a roughly twofold increase since early 2025. 2025.
- [JED20] JEDEC Solid State Technology Association. *DDR5 SDRAM Standard (JESD79-5)*. Verified 2026-05-05 (JEDEC’s catalogue page returned 403 to unauthenticated fetches; first-publication date confirmed via the JEDEC press release dated 14 July 2020 and corroborated through Synopsys and VideoCardz coverage). The standard has since been revised (JESD79-5A, -5B, -5C, -5D); the citation refers to the DDR5 standard family. Cited in §6.1 to denote the DDR5 main-memory standard the test machine implements. July 2020.
- [Jia+25] F. Jia, Z. Wu, S. Jiang, H. Jiang, Q. Zhang, Y. Yang, Y. Liu, J. Ren, D. Zhang, and T. Cao. *Scaling Up On-Device LLMs via Active-Weight Swapping Between DRAM and Flash*. Verified 2026-05-05 against arXiv 2504.08378 (v1: 11 Apr 2025; v2: 23 Sep 2025). All ten authors listed; venue is arXiv preprint, not yet a conference proceeding. The abstract supports the three techniques we cite in §3.1: predicting next-layer active weights from the current layer’s activations, sparsity-aware fine-tuning that aligns the active-weight pattern with the dense-model output, and a dynamic DRAM–flash swap pipeline at runtime. Apr. 2025. arXiv: 2504.08378 [cs.LG].
- [Kam+25] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci. “Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models.” In: *International Conference on Learning Representations (ICLR)*. Verified 2026-05-05 against arXiv 2402.07033 (submitted 10 Feb 2024). The previous bib entry was missing co-author Tian Tang and listed the work as a 2024 arXiv preprint; the paper was accepted to ICLR 2025. Cited in §3.2 as the closest CPU–GPU MoE design in spirit; the specific weight-transfer-vs-activation-shipping decision rule we summarise is in the body of the paper, not the abstract. 2025. arXiv: 2402.07033 [cs.LG].
- [KL25a] M. Kerrisk and Linux man-pages project. *mlock(2): lock and unlock memory (mlock, mlock2, munlock, mlockall, munlockall)*. Linux man-pages 6.16. Verified 2026-05-05. The page documents the `MLOCK_ONFAULT` flag we cite in §7.2: “Lock pages that are currently resident and mark the entire range so

- that the remaining nonresident pages are locked when they are populated by a page fault.” Oct. 2025.
- [KL25b] M. Kerrisk and Linux man-pages project. *mmap(2): Linux manual page*. Linux man-pages 6.16. Verified 2026-05-05. Maintained by Michael Kerrisk for the Linux man-pages project. Cited in §2.3 for the file-into-address-space semantics that llama.cpp’s default loader relies on. Oct. 2025.
- [KL25c] M. Kerrisk and Linux man-pages project. *open(2): Linux manual page*. Linux man-pages 6.16. Verified 2026-05-05. The page’s `O_DIRECT` section states that since Linux 2.6.0 the file offset, request length, and buffer address must be multiples of the block device’s logical block size (typically 512 bytes), which is the alignment contract we cite in §2.4. Oct. 2025.
- [Liu+23] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Ré, and B. Chen. “Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time.” In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202:22137–22176. Verified 2026-05-05 against arXiv 2310.17157 and the PMLR proceedings page. Published at ICML 2023 as an oral presentation; the arXiv submission (Oct 2023) post-dates the conference. Authors confirmed. Cited in §3.1 as the comparator for activation-sparsity exploitation in dense LLMs. 2023. arXiv: 2310.17157 [cs.LG].
- [Mir+24] I. Mirzadeh, K. Alizadeh, S. Mehta, C. C. Del Mundo, O. Tuzel, G. Samei, M. Rastegari, and M. Farajtabar. “ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models.” In: *International Conference on Learning Representations (ICLR)*. Verified 2026-05-05 against arXiv 2310.04564 (submitted 6 Oct 2023) and the OpenReview record. Originally a 2023 preprint; published as an ICLR 2024 conference paper (oral). Apple authorship. Cited in §3.1 for the high post-activation sparsity (~90%) that ReLU FFNs exhibit; that figure is in the body of the paper rather than the abstract. 2024. arXiv: 2310.04564 [cs.LG].
- [MM03] N. Megiddo and D. S. Modha. “ARC: A Self-Tuning, Low Overhead Replacement Cache.” In: *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST ’03)*. Verified 2026-05-05 against the USENIX FAST ’03 conference page. Authors and venue confirmed; previous bib entry used `@article` with `journal = FAST`, which is structurally wrong because FAST is a conference. Corrected to `@inproceedings`. Cited in §4.4 (tracer simulator chapter) for the Adaptive Replacement Cache algorithm. Mar. 2003.

- [NVI26] NVIDIA Corporation. *GPUDirect Storage Documentation*. Verified 2026-05-05. Official NVIDIA documentation portal (copyright 2020–2026). Cited in §7.2 (future work) for the technology that enables direct DMA between storage and GPU-resident memory, bypassing CPU buffers; the same SQE–DMA pattern we use today on host buffers could in principle target GPU buffers via this facility. 2026.
- [NVM25] NVM Express Consortium. *NVM Express Base Specification (Revision 2.3)*. Verified 2026-05-05 against the official NVMe specifications portal. The current revision (2.3) was released on 5 August 2025 by the NVM Express non-profit consortium that defines the NVMe family of specifications (Base, Command Set, Transport, Management). Cited in §2.4 for the concept of a per-device logical block size; the specific 512-byte value mentioned in that sentence is a property of the Samsung 980 PRO used in our setup, not a constant of the NVMe spec. Aug. 2025.
- [Ope25] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. Verified 2026-05-05 against the arXiv PDF (v1, document dated August 5, 2025; arXiv submission August 8, 2025). Section 2.2 supports every architectural claim we cite: “gpt-oss-120b, which consists of 36 layers ... gpt-oss-20b with 24 layers” (§2 opening); “Each MoE block consists of a fixed number of experts (128 for gpt-oss-120b and 32 for gpt-oss-20b) ... we select the top-4 experts for each token” (§2.2 Mixture-of-Experts); “The MoE blocks use the gated SwiGLU activation function”; “attention blocks alternate between banded window and fully dense patterns ... the bandwidth is 128 tokens” (§2.2 Attention). MoE weights are MXFP4 at 4.25 bits per parameter, which yields the 4.20 MiB per expert weight matrix that we measure in the GGUF file. Aug. 2025. arXiv: 2508.10925 [cs.CL].
- [PCI17] PCI-SIG. *PCI Express Base Specification, Revision 4.0, Version 1.0*. Verified 2026-05-05 (corroborated via secondary sources; the pcisig.com members-only portal blocks unauthenticated fetches). Released by PCI-SIG on 8 June 2017. Cited in §6.1 to characterise the PCIe 4.0 link of the Samsung 980 PRO used as the model store. June 2017.
- [Raj+22] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He. “DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale.” In: *International Conference on Machine Learning (ICML)*. Verified 2026-05-05 against arXiv 2201.05596. Title, authors, and ICML 2022 publication confirmed on the arXiv listing. Cited in §3.2 for MoE training and inference at GPU-cluster scale, including the PR-MoE compression scheme; specific design details

- (expert parallelism with cross-GPU routed activations, tensor parallelism for dense layers) are in the body of the paper rather than the abstract. 2022. arXiv: 2201.05596 [cs.LG].
- [Sha+17] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.” In: *International Conference on Learning Representations (ICLR)*. Verified 2026-05-05 against the arXiv listing (submitted 23 January 2017). Authors and title confirmed. Original sparsely-gated mixture-of-experts paper, cited in §2.2 and §1 for the MoE concept. 2017. arXiv: 1701.06538 [cs.LG].
- [Sha20] N. Shazeer. *GLU Variants Improve Transformer*. Verified 2026-05-05 against arXiv 2002.05202. Single author. Studies several gated-linear-unit variants for the transformer feed-forward block, including SwiGLU (the Swish-GLU combination), which the GPT-OSS model card cites as the activation it uses. Cited in §3.1 to identify the gated rather than thresholded character of GPT-OSS’s FFN, distinguishing it from the ReLU-sparsity assumption that LLM-in-a-Flash relies on. Feb. 2020. arXiv: 2002.05202 [cs.LG].
- [She+23] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang. “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU.” In: *International Conference on Machine Learning (ICML)*. Verified 2026-05-05 against arXiv 2303.06865 (v1: 13 Mar 2023; v2: 12 Jun 2023). The previous bib entry listed only ten of the fourteen authors and was missing Daniel Y. Fu, Zhiqiang Xie, Clark Barrett, and Joseph E. Gonzalez. The abstract supports our §3.1 claims: aggregating memory and compute across GPU, CPU and disk, and using a linear-programming search to pick a per-tensor placement policy. Conference publication at ICML 2023 is widely attested but is not visible on the arXiv abstract page. 2023. arXiv: 2303.06865 [cs.LG].
- [Son+24] Y. Song, Z. Mi, H. Xie, and H. Chen. “PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU.” In: *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP)*. Verified 2026-05-05 against arXiv 2312.12456 (v1: 16 Dec 2023; v2: 12 Dec 2024). Published at SOSP 2024. Authors confirmed. The abstract directly supports the hot-vs-cold neuron classification and the GPU-pin-plus-CPU-compute split that we cite in §3.1. 2024. arXiv: 2312.12456 [cs.LG].

- [SPD26] SPDK Project. *SPDK: Storage Performance Development Kit*. Verified 2026-05-05. Open-source project providing user-mode NVMe drivers with polled, lockless, thread-per-core completions; originally sponsored by Intel, transitioning to the Linux Foundation in 2025. BSD 3-Clause licensed. The most recent release listed on the homepage at verification time was dated 2026-01-28. Cited in §3.3 as the reference user-space storage stack against which we contrast the kernel-mediated `io_uring + 0_DIRECT` path used here. 2026.
- [Swa25] G. Swain. *Server memory prices could double by 2026 as AI demand strains supply*. Network World. Verified 2026-05-05: Network World, citing Counterpoint Research. Reports that DDR5 64 GB RDIMM modules could cost roughly twice as much by the end of 2026 as at the start of 2025. Nov. 2025.
- [Tre25] TrendForce. *[News] AI Reportedly to Consume 20% of Global DRAM Wafer Capacity in 2026, HBM and GDDR7 Lead Demand*. Verified 2026-05-05: TrendForce News, citing Commercial Times. Article supports the claim that AI consumption will reach about 20% of global DRAM supply in 2026; it does not directly address DDR5 price doubling. Dec. 2025.
- [Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention Is All You Need.” In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Verified 2026-05-05 against the NeurIPS proceedings page. Original transformer paper. Cited in §2.1 for the layered transformer architecture. 2017.
- [Xue+24a] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. K. Marina. *MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache*. Verified 2026-05-05 against arXiv 2401.14361. The paper has been revised three times; the title cited here is the v3 (12 Mar 2025) title. Prior versions used “Activation-Aware Expert Offloading for Efficient MoE Serving” (v1, 25 Jan 2024) and “Offloading-Efficient MoE Model Serving” (v2). Cited in §3.2 and §7.2 (future work) for the Expert Activation Matrix (EAM) construction and the cosine-similarity request-matching prefetch policy; both are described in the body of the paper rather than the abstract, but are well-attested in the literature on this work. Jan. 2024. arXiv: 2401.14361 [cs.LG].
- [Xue+24b] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen. *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. Verified 2026-05-05 against arXiv 2406.06282 (v1: 10 Jun 2024; latest revision 12 Dec 2024). All six authors listed; venue is arXiv preprint, no conference proceedings

indexed. The abstract supports our §3.1 claim that the work extends the neuron-cluster decomposition to a smartphone NPU plus CPU split (dense neuron clusters on the NPU, sparse clusters on the CPU). June 2024. arXiv: 2406.06282 [cs.LG].